Lab 2：BombLab 实验报告

2026 年 10 月 13 日

摘要

本实验针对个人 64 位 bomb，依据反汇编和只读数据区恢复每关的输入校验关系，并在 `defuser.c` 中实现由 64 位 `challenge` 计算答案的求解函数。报告记录六个常规关卡与 Honor 隐藏关的主要计算关系、验证方法、结果截图和辅助工具使用情况。答案按 challenge 计算，不依赖某个固定 seed 对应的答案表。

1 实验目标与分析方法

每个常规关卡接收一个 64 位 `challenge`，bomb 将输入行解析为一个或多个无符号整数，再与由 challenge 计算出的目标值比较。Honor 关还会接收前六关答案行生成的 `context`。因此，求解程序要实现的是计算规则，而不是记忆一次运行的口令。

阅读反汇编时，我重点查看了输入解析函数的参数数量、challenge 被移位或掩码后送入哪些寄存器、条件跳转选择的分支，以及最终参与比较的返回值。遇到跳转表、函数指针表或链表时，还结合了 `.rodata` 中的数据和实际指针关系。报告中的常量和关系来自仓库个人 bomb 的反汇编与数据区。

代码中的无符号整数运算按 32 位结果理解；加法和乘法溢出会保留低 32 位。符号如下：

$$\mathrm{lo}_{32}(z) = z \bmod 2^{32}.$$

2 六个常规关卡

2.1 第一关：参数传递与组合计算

`phase_01` 解析一个整数，并将它与 `p1_compute(challenge)` 的返回值比较。计算函数从 challenge 中抽取低 32 位、高 32 位和右移 16 位后的低 32 位；三个辅助函数中的 `lea` 完成乘加，`xor` 完成按位异或。

令

$$L = \mathrm{lo}_{32}(challenge), \quad H = \mathrm{lo}_{32}(challenge \gg 32), \quad M = \mathrm{lo}_{32}(challenge \gg 16).$$

中间量和答案为

$$t_2 = (5H + L)\ \texttt{xor}\ \texttt{0x6d13},$$
$$t_0 = (L + 8M)\ \texttt{xor}\ \texttt{0xe797},$$
$$answer = (t_2 + \texttt{0xc884})\ \texttt{xor}\ t_0.$$

本关输出一个无符号十进制整数。这里的关键是按调用顺序追踪寄存器参数和返回值，而不是把 challenge 当作一个整体直接取高位或低位。

2.2 第二关：数组与递推

`phase_02` 要求成功解析六个整数。首项来自 challenge 右移 40 位后的低位；后五项在循环中由前一项递推。令 $v_0, \ldots, v_5$ 为输入，则

$$v_0 = \big((2 \log_{32}(challenge \gg 40))\ \texttt{and}\ \texttt{0x1e}\big) + 2,$$
$$p_i = \log_{32}(challenge \gg (i + 18))\ \texttt{and}\ \texttt{0x1f},$$
$$v_i = (p_i + v_{i-1})\ \texttt{xor}\ (6i), \qquad i = 1, \ldots, 5.$$

输出顺序与数组顺序相同。反汇编中的循环计数从 1 递增到 5；追踪该计数器可以确定右移量和异或项中的 i。

2.3 第三关：switch 选择与跳转表

`phase_03` 解析两个整数。第一个整数必须等于 challenge 的最低三位；该值再索引跳转表，决定第二个整数应等于哪一个计算函数的返回值。令

$$s = challenge\ \texttt{and}\ 7, \qquad x = \log_{32}(challenge \gg 32)\ \texttt{xor}\ \log_{32}(challenge \gg 3).$$

根据跳转表地址和各个 `p3_case` 函数，selector 与结果的对应关系如下。

s	第二个整数的计算式
0	$5x + \texttt{0x436de}$
1	$(x\ \texttt{xor}\ \texttt{0xfc30}) + (x \gg 2)$
2	$5x + \texttt{0xd48a}$
3	$5x + \texttt{0xc723}$
4	$5x + \texttt{0x5b36}$
5	$(2x)\ \texttt{xor}\ \texttt{0x8d1f}$
6	$(x\ \texttt{xor}\ \texttt{0x113f}) + (x \gg 1)$
7	$(2x)\ \texttt{xor}\ \texttt{0x11f9}$

跳转表中的地址经由短跳板再到达 case 函数，所以不能仅按函数在反汇编中的排列顺序建立 selector 映射。最终输出 s 和对应计算结果两个数。

2.4 第四关：递归二分查找与回溯

`phase_04` 解析一个整数，与递归函数 `p4_walk` 的返回值比较。challenge 的低 7 位给出二分查找目标，范围固定为 $[0, 127]$。每层先计算

$$mid = low + \left\lfloor \frac{high - low}{2} \right\rfloor, \quad target = \mathrm{lo}_{32}(challenge) \texttt{ and } \texttt{0x7f},$$

而盐值为

$$salt = \mathrm{lo}_{32}(challenge \gg 7) \texttt{ xor } \mathrm{lo}_{32}(challenge \gg 32).$$

命中中点时返回 $(mid + \texttt{0x31}) \texttt{ xor } salt$。递归返回阶段还会把子调用结果乘 3、与 $mid + salt$ 异或，再与分支常量异或：

$$target < mid: \quad result = (3 \cdot result \texttt{ xor } (mid + salt)) \texttt{ xor } \texttt{0x0c},$$

$$target > mid: \quad result = (3 \cdot result \texttt{ xor } (mid + salt)) \texttt{ xor } \texttt{0x15}.$$

实现时先沿二分路径记录各层的中点和分支，再从最深层向上应用返回组合，等价于原递归函数逐层返回。

2.5 第五关：结构体、指针表与链表遍历

`p5_traverse` 使用一组 16 字节节点。由节点访问偏移可恢复出如下布局：

```
struct Node {
    struct Node *next;   /* offset 0, 8 bytes */
    uint32_t value;      /* offset 8 */
    uint32_t tag;        /* offset 12 */
};
```

起始指针表由 challenge 最低三位索引；$(challenge \gg 3) \texttt{ and } 3$ 给出沿 `next` 指针前进的步数。指针表的排列与节点在内存中的排列不同，遍历必须跟随 `next` 字段。

令起始节点为 n_0，并设

$$h = (challenge \gg 3) \texttt{ and } 3,$$

$$r_0 = \big(\mathrm{lo}_{32}(challenge \gg 6) \texttt{ xor } \mathrm{lo}_{32}(challenge \gg 32) \texttt{ xor } tag(n_0) \texttt{ xor } \texttt{0x9843}\big) + value(n_0).$$

之后对 $j = 1, \ldots, h$：

$$n_j = n_{j-1}.\texttt{next}, \qquad r_j = (r_{j-1} \texttt{ xor } tag(n_j)) + value(n_j).$$

两个输入依次为最终累计值 r_h 和最后访问节点的 `value`；若 $h = 0$，最后访问节点就是起始节点。

2.6 第六关：函数指针表与间接调用

`phase_06` 解析一个整数。初始值和两个 selector 为

$$x = \mathrm{lo}_{32}(challenge \gg 6) \texttt{ xor } \mathrm{lo}_{32}(challenge \gg 32),$$

$$i_1 = challenge \texttt{ and } 7,$$

$$i_2 = (challenge \gg 3) \texttt{ and } 7.$$

函数指针表将 selector 映射到操作编号：

$$[2, 2, 0, 4, 5, 3, 1, 3].$$

先执行 i_1 选择的操作，随后异或 `0x214e`，再执行 i_2 选择的操作，最后加 `0x3c60`。各操作如下，所有结果都截取为 32 位：

操作编号	32 位运算
0	循环右移 7 位 $\mathrm{ror}_{32}(x, 7)$
1	$(x \texttt{ xor } \texttt{0x906b}) + (x \ll 7)$
2	$((x \gg 1) \texttt{ xor } x) + \texttt{0x56e9}$
3	$(4x + \texttt{0x6079}) \texttt{ xor } (3x)$
4	$(x + \texttt{0x4228}) \texttt{ xor } (x \gg 1)$
5	$\mathrm{rol}_{32}(x, 1) \texttt{ xor } \texttt{0x74cd}$

循环移位与普通逻辑移位不同：移出边界的位会从另一端绕回。根据反汇编的移位、异或和加法顺序逐个还原六个操作，再按函数指针表选择操作，是本关的关键。

3 Honor 隐藏关

`secret_gate` 检查输入行偏移 32 处是否为字符串 `OPEN`，随后解析三个无符号整数。程序先输出 32 个字符的前缀，再输出 `OPEN` 和三个整数；前缀长度及格式必须同时满足门禁检查。

定义

$$\begin{aligned} C &= \mathrm{lo}_{32}(\mathit{challenge}) \texttt{ xor } \mathrm{lo}_{32}(\mathit{challenge} \gg 32), \\ K &= \mathrm{lo}_{32}(\mathit{context}) \texttt{ xor } \mathrm{lo}_{32}(\mathit{context} \gg 32), \\ r &= \mathrm{lo}_{32}(\mathit{context}) \texttt{ and } 3. \end{aligned}$$

`secret_mix(z)` 先将 32 位 z 循环左移 3 位，再加 `0x9555`，最后异或 `0xd26d`。三项数值为

$$\begin{aligned} \mathit{first} &= r, \\ \mathit{second} &= \texttt{secret_mix}(C \texttt{ xor } K) \texttt{ xor } \big((\texttt{0x9555}r) \texttt{ xor } \mathrm{lo}_{32}(\mathit{context} \gg 16)\big), \\ \mathit{third} &= \mathit{second} \texttt{ xor } \big(K + r + \texttt{0xd26d} + (C \gg 3)\big). \end{aligned}$$

`context` 由同一 seed 下前六关的答案行生成，因此 Honor 必须与这六行一并验证。

4 验证结果

编译使用仓库 Makefile 中配置的 C11、优化、警告和 `-Werror` 选项。报告记录的验证结果如下：

验证项目	结果
六个常规关卡的公开测试	60/60 通过
Honor 的公开测试	10/10 通过（可选测试集）
非公开 seed 1337 的六关及 Honor 完整流程	通过
固定随机源 `random.Random(0xB0B)` 生成的 128 个 64 位 seed	128/128 完整流程通过

公开测试截图中的 60/60 与 10/10 是仓库本地测试脚本的结果。seed 1337 和额外 128 个随机 seed 属于本地补充检查，不是课程隐藏测试，也不能替代隐藏评分。完整流程测试会先生成六行答案并交给 bomb，再基于相同运行上下文生成并验证 Honor 输入。

复现公开测试的命令：

```
make
python3 public_test.py
```

课程隐藏 challenge 和 seed 未在本地运行。

5 运行与测试截图

图 1 展示个人 bomb 的六个常规关卡全部解除以及最终结局提示。图 2 展示 `make` 后运行公开测试的终端输出，其中六个常规关卡合计 60/60，Honor 公开测试为 10/10。



图 1: 个人 bomb 六个常规关卡全部解除后的终端结果。

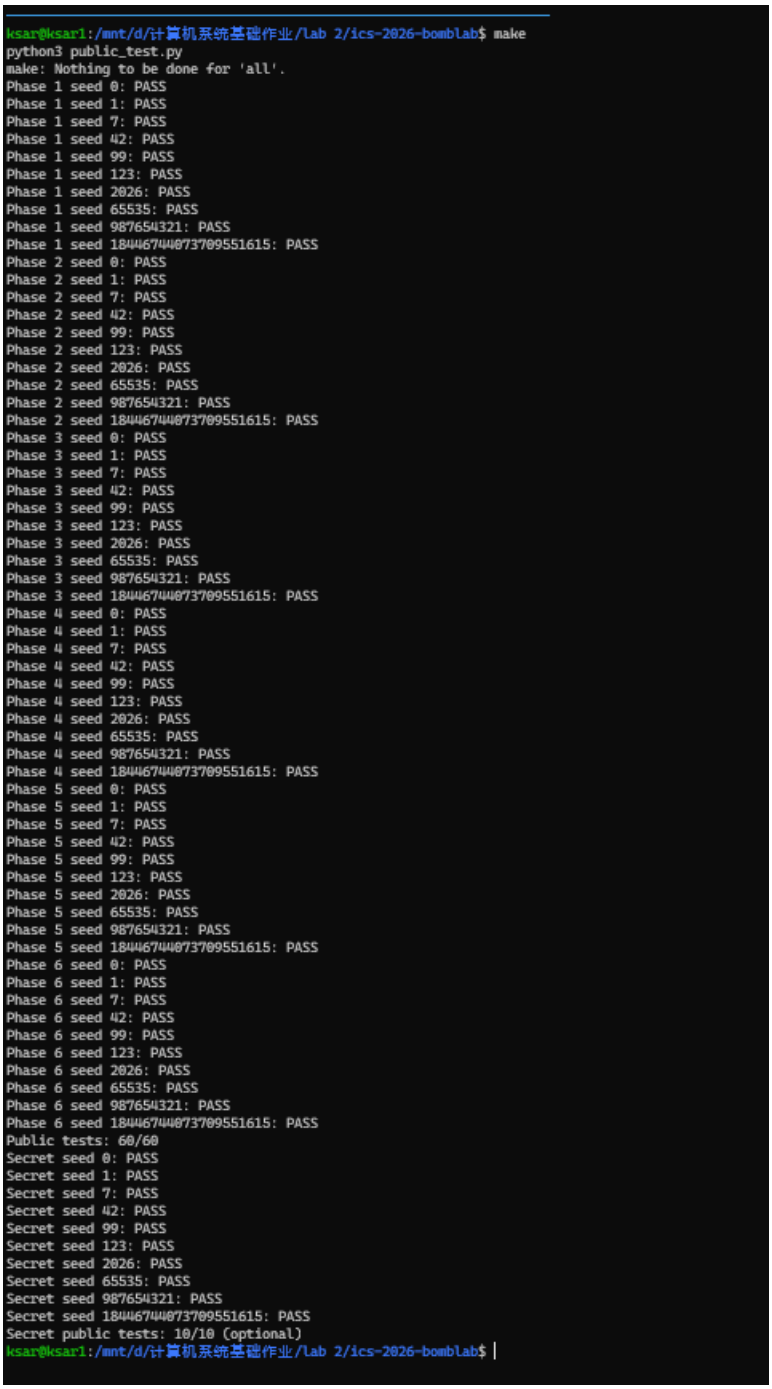


图 2: 编译状态与公开测试结果：常规关卡 60/60，Honor 10/10。

6 AI 与其他辅助工具使用说明

本报告文本由 OpenAI Codex 协助起草、重写与整理。Codex 也用于阅读仓库说明、协助运行编译与测试。AI 参与了验证检查代码的过程。本人对最终提交内容负责，并能说明各关卡的关键计算关系。

引用和使用的资料包括：课程仓库中的 `README.md` 与测试脚本、个人 bomb 的 `objdump` 反汇编及 `.rodata` 数据，以及 GCC、GNU binutils 和 GDB 等课程实验工具。